

Assignment 2: Problem Statement

1 Introduction

The goal of this assignment is to gain hands-on experience with sequential decision-making problems.

The assignment is divided into two parts. Part A concerns modeling a decision-making problem as an MDP and then determining the optimal policy. Here, the agent will possess knowledge of the transition model and reward model that will be used in the process of obtaining the optimal policy that maximizes the cumulative reward.

In Part B, we will work with a domain where the transition model and the reward model will not be a-priori specified to the agent. Hence, the agent would need to interact with the environment to gather experience and then learn a good policy via reinforcement learning (RL). The assignment involves learning a classic RL algorithm called Q-learning for a problem that involves control of a car in a busy environment.

This assignment considers the tabular setting (discrete state and action spaces) for both domains. Although the problems are situated in toy domains, but can be conceptually extrapolated to logistics/delivery-style problems or problems that involving controlling a system.

2 Problem Setting

2.1 Part A: Determining the Optimal Policy for an MDP Environment

In Part A, you will work with the **TreasureHunt** environment. In this environment, the agent controls a ship navigating through the sea. The objective of the agent is to navigate the ship to a fort while avoiding pirates and collecting treasures along the way. Due to winds, the execution of the agent's actions is stochastic and the transition dynamics governing this uncertainty are known to the agent. The domain is described below. Note: the transition model and the reward model will be provided to you in a file and can be used to determine an effective policy for this domain.

2.1.1 Layout

The sea is represented as an $N \times N$ grid. Each position in the grid is represented by a tuple (i, j) , where $i, j \in 0, \dots, N - 1$. The position $(0, 0)$ corresponds to the bottom-left corner of the grid.

The position of the ship is denoted by S . Each grid cell may contain one of the following: land, a treasure, the fort, a pirate, or empty sea.

There are exactly two pirates, whose locations are denoted by P_1 and P_2 , and exactly two treasures, whose locations are denoted by T_1 and T_2 . The location of the fort is denoted by F . Refer to Figure 1 for an illustration of the environment.

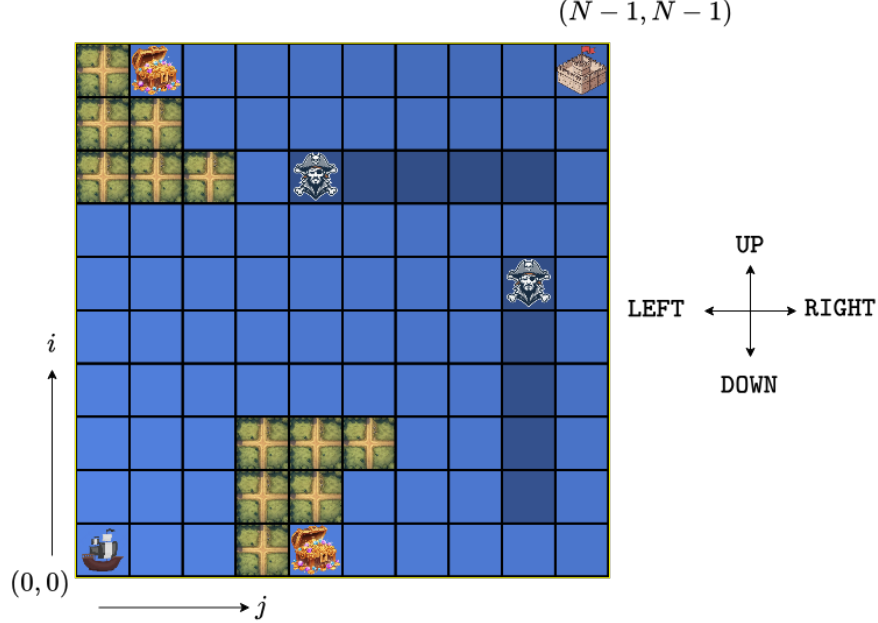


Figure 1: An instance of the TreasureHunt environment. The treasure, the fort and the land, empty sea are shown. The dark blue region relates to the cells where the pirates can move. The agent can take four actions that are shown on the right.

2.1.2 State Representation

At each time step, the agent observes the state of the environment, which consists of the current location of the ship, the locations of the two pirates, and the locations of the treasures that have not yet been collected. The state can be represented as:

$$x = (S, P_1, P_2, \mathcal{T}),$$

where S denotes the current location of the ship, P_1 and P_2 denote the current locations of the two pirates, and $\mathcal{T}$ denotes the set of locations of the remaining treasures. Initially, $\mathcal{T}$ contains the locations of both treasures. When the ship collects a treasure, that treasure is removed from $\mathcal{T}$. Therefore, $\mathcal{T}$ may contain two, one, or zero treasure locations.

2.1.3 Actions and Transition Model

The agent's action space is discrete and described below. Note that actions when executed have uncertain outcomes. The uncertainty is captured in the transition model (described below) that captures the distribution over the next state given the current state and the action to be taken.

- The agent controls the ship by selecting one of the actions $\{\text{UP}, \text{DOWN}, \text{LEFT}, \text{RIGHT}\}$. Each action attempts to move the ship to a neighbouring grid cell. For example, if $S = (i, j)$, the UP action attempts to move the ship to $(i + 1, j)$, provided that $i + 1 < N$. Refer to Figure 1 for the directions corresponding to the other actions.
- If an action would cause the ship to move outside the grid or into a cell containing land, the ship remains in its current grid cell.

- Due to wind, the intended action may not always be executed successfully. The intended action is executed with probability p_s . With probability $1 - p_s$, one of the remaining three actions is executed uniformly at random. Therefore, each of the remaining actions is executed with probability $\frac{1-p_s}{3}$.
- If the ship moves to a grid cell containing a treasure, the treasure is collected and removed from the environment. If the ship moves to the fort, the episode terminates successfully. If the ship moves to a grid cell containing a pirate, the episode also terminates.
- In addition to the ship, the pirates move within their respective regions. Each pirate is restricted to a connected region of the grid, and the regions of the two pirates are disjoint. Unlike the ship, the movements of the pirates cannot be controlled by the agent.

Each pirate independently selects one of the actions {UP, DOWN, LEFT, RIGHT}. Pirate P_1 selects these actions with probabilities $p_{1,u}$, $p_{1,d}$, $p_{1,l}$, $p_{1,r}$, respectively. Similarly, pirate P_2 selects its actions with probabilities $p_{2,u}$, $p_{2,d}$, $p_{2,l}$, $p_{2,r}$, respectively. If a selected action would cause a pirate to move outside its allowed region, the pirate remains in its current grid cell.

2.1.4 Reward Model

- The agent receives a step reward of $R_s < 0$ at every time step. If the ship moves to a cell containing a treasure, it receives an additional reward of $R_t > 0$.
- If the ship reaches the fort, it receives an additional reward of $R_f > 0$, so the total reward for that time step is $R_s + R_f$. If the ship moves to a cell containing a pirate, it receives an additional reward of $R_p < 0$, so the total reward for that time step is $R_s + R_p$.
- As in a standard MDP, the rewards obtained along a trajectory are discounted using the discount factor γ . Thus, the objective of the agent is to maximize the expected cumulative discounted reward (return).

2.1.5 Implementation

- Formalize this problem as an MDP. Note that there are complex interactions in the domain. The agent is to move and collect treasures but must also deal with pirates. Develop an intuition into what a good policy will be for such domains.
- Next, implement a method to determine the policy that maximizes the expected cumulative reward. Note that the agent has knowledge of the transition and the reward models. Refer to the class discussion on ways to determine the optimal policy in an MDP. See implementation hints for getting started 3.5
- Please see the details of the input/output format and evaluation procedure provided in the Submission and Evaluation section 3.2.1. Examine the behavior of your policy by running it in the environment.

2.2 Part B: Tabular Reinforcement Learning

In Part B, we consider the **Highway** environment. Here, the agent is a autonomous car which is learning to drive safely on a highway. There are simulated cars on the highway. The objective of the agent is to drive the control car as far as possible along the highway while avoiding collisions

with other cars. Unlike Part A, the knowledge of the transition dynamics and reward function of the environment is not available to the agent. The agent must interact with the environment and then learn a good policy. The environment is described below.

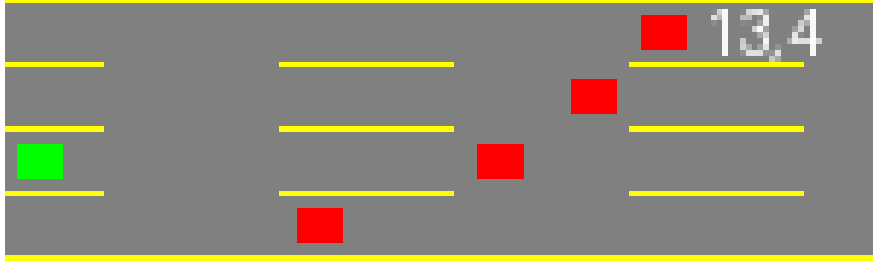


Figure 2: An instance of the Highway environment. The control car is shown in green, while the other simulated cars are shown in red. The highway consists of four lanes.

2.2.1 Layout

The environment consists of a control car and other simulated cars moving along a highway with four lanes. The control car, shown in green, is controlled by the agent, while the other cars, shown in red, are controlled by the environment. The agent can control both the speed of the control car and the lane in which it is driving. The objective is to drive the control car as far as possible without colliding with any other car. An illustration of the environment is shown in Figure 2.

2.2.2 State Representation

At each time step t , the agent observes the state $x_t = (s_t, l_t, d_t^1, d_t^2, d_t^3, d_t^4)$, where $s_t \in \{1, 2, 3, 4\}$ denotes the discrete speed of the control car and $l_t \in \{1, 2, 3, 4\}$ denotes the lane in which the control car is currently driving.

For each lane $i \in \{1, 2, 3, 4\}$, the value $d_t^i \in \{0, 1, 2, 3, 4\}$ denotes the discretized distance from the control car to the closest car ahead in lane i . The distance is measured along the direction of the highway and is discretized into five possible values ranging from 0 to 4.

2.2.3 Actions

At each time step, the agent can select one of the following actions:

$$a_t \in \{\text{inc speed}, \text{dec speed}, \text{inc lane}, \text{dec lane}, \text{no op}\}.$$

The effects of these actions on the speed and lane of the control car are described below:

- If $a_t = \text{inc speed}$, the speed is increased by one: $s_{t+1} = \min(s_t + 1, 4)$.
- If $a_t = \text{dec speed}$, the speed is decreased by one: $s_{t+1} = \max(s_t - 1, 1)$.
- If $a_t = \text{inc lane}$, the control car moves to the next lane: $l_{t+1} = \min(l_t + 1, 4)$.
- If $a_t = \text{dec lane}$, the control car moves to the previous lane: $l_{t+1} = \max(l_t - 1, 1)$.
- If $a_t = \text{no op}$, the speed and lane of the control car are unchanged: $s_{t+1} = s_t$, and $l_{t+1} = l_t$.

2.2.4 Transition Model

The evolution of the other cars and the resulting state is by simulating the transition model. The simulator takes in the action and outputs the next state to the agent. The inherent uncertainty in action outcomes is not known to the agent.

2.2.5 Reward Model

After each transition, the agent receives an instantaneous reward from the simulated environment. The reward function used by the environment is:

$$r(x_t, a_t, x_{t+1}) = \begin{cases} -5, & \text{if the control car collides with another car,} \\ 0.03 s_{t+1}, & \text{otherwise.} \end{cases}$$

The rewards for the trajectory are discounted by the discount factor γ .

Note: the explicit form of the reward function is not provided to the agent. The agent receives the reward emitted by the simulator and then learns from it.

2.2.6 Implementation

- The goal is to train the agent to navigate on the highway and collect good rewards. The learning is to be with interacting with the environment which will reveal the rewards and the inherent action uncertainty. Think of the problem as an MDP where the reward and transition models are not known.
- Next, implement a Q-learning agent (<https://en.wikipedia.org/wiki/Q-learning>) that learns a policy that aims to maximize the expected cumulative reward while avoiding collisions. This is a classic RL algorithm. The learning is via interaction with the environment/simulator. We will use it in the tabular setting. Refer to the implementation hints for getting started 3.5
- Spend time in examining the behavior of the learnt policy when it is executed in the simulator. Analyze if the policy behaviour feels intuitive and how it responds to various states of the Highway that may come up in simulations.
- Please see the details of the input/output format and evaluation procedure provided in the Submission and Evaluation section 3.2.2.

3 Submission Guidelines

As this is a large class, the assignments will be evaluated automatically. Therefore, please follow the submission guidelines carefully. *Deviation(s) from the specified format may cause the entire evaluation process to stall and may result in a penalty.*

3.1 Starter Code and Environment Setup

You are provided with the starter code on Moodle. The structure of the starter code is as follows:

```
A2-starter-code/  
├── requirements.txt  
├── part_a/  
│   ├── env.py  
│   ├── run.py  
│   ├── agent.py  
│   ├── images/  
│   └── tests/  
│       ├── 1/  
│       │   ├── layout.txt  
│       │   └── prob.txt  
│       ├── 2/  
│       └── 3/  
└── part_b/  
    ├── env.py  
    ├── run.py  
    └── agent.py
```

You should modify only the files highlighted in red and will submitting these files only along with the report and group file. Your solution for Part A should be implemented in `part_a/agent.py`, while your solution for Part B should be implemented in `part_b/agent.py`. The purpose of the remaining files will be explained in the subsequent sections.

Your code will be evaluated in a protected Conda environment. Therefore, make sure that your code runs correctly in the following environment:

```
conda create -n ai_a2 python=3.10  
conda activate ai_a2  
pip install -r requirements.txt
```

Please do not use any additional Python packages. The evaluation will be performed on a single CPU machine with 64 GB of RAM. **Do not use multithreading, multiprocessing, or subprocesses in your code.**

3.2 Input/Output Format

3.2.1 Part A

For Part A, your program will be run using the following command:

```
python run.py --layout_file <layout file path> --prob_file <prob file path> \
--num_runs 10 --T 120 --output_dir <output path>
```

For example:

```
python run.py --layout_file tests/1/layout.txt --prob_file tests/1/prob.txt \
--num_runs 10 --T 120 --output_dir ./outputs/1
```

The `layout_file` specifies the layout of the environment, while the `prob_file` specifies the transition probabilities and rewards. These files are described in more detail below.

The parameter `T` specifies the time allocated for training, `num_runs` specifies the number of trajectories used to evaluate your learned policy, and `output_dir` specifies the directory in which the visualizations and outputs will be stored.

Internally, `run.py` initializes your agent using the layout and probability files. It then calls the `learn_policy` function with `T` as input. Your agent must learn its policy within the specified time limit. After training, the learned policy is evaluated by repeatedly calling the `get_action` function.

Layout File. The layout file specifies the initial state and layout of the grid. It contains N lines, with each line containing N characters. The j -th character of the i -th line specifies the content of the corresponding grid cell.

For example, the layout corresponding to the environment shown in Figure 1 is:

```
SWWLTWWWWW
WWWLLWW!W
WWWLLLWW!W
WWWWWWW!W
WWWWWWW!W
WWWWWWW1W
WWWWWWWWW
LLLW2!!!W
LLWWWWWWW
LTWWWWWWF
```

In this example, $N = 10$. The symbol `S` denotes the initial location of the ship, `W` denotes an empty sea cell, `L` denotes land, `T` denotes a treasure, and `F` denotes the fort. The symbols `1` and `2` denote the initial locations of Pirate 1 and Pirate 2, respectively, while `!` denotes a cell belonging to a pirate's movement region.

Probability and Reward File. The probability file specifies the transition probabilities of the environment and the rewards:

```

0.9
0.5 0.5 0.0 0.0
0.0 0.0 0.5 0.5
-0.05 3 5 -1
0.99

```

The first line contains the probability p_s that the intended action of the ship is executed successfully. The second line contains the action probabilities for Pirate 1, namely $p_{1,u}, p_{1,d}, p_{1,l}, p_{1,r}$, corresponding to the actions UP, DOWN, LEFT, and RIGHT, respectively. For example, in the configuration above, $p_{1,u} = 0.5$, $p_{1,d} = 0.5$, $p_{1,l} = 0$, and $p_{1,r} = 0$. Similarly, the third line contains the action probabilities for Pirate 2, namely $p_{2,u}, p_{2,d}, p_{2,l}, p_{2,r}$. The fourth line contains the rewards R_s, R_t, R_f , and R_p , corresponding to the step reward, treasure reward, fort reward, and pirate reward, respectively. The final line contains the discount factor γ .

3.2.2 Part B

For Part B, your program will be run using the following command:

```
python run.py --num_runs 10 --T 120 --output_dir <output path> --df 0.99
```

The parameter `T` specifies the time allocated for training, `num_runs` specifies the number of trajectories used to evaluate the learned policy, and `output_dir` specifies the directory in which the visualizations and outputs will be stored. The discount factor γ is specified using the `df` argument.

Internally, `run.py` initializes your agent and calls the `learn_policy` function with `T` as input. Your agent must learn a policy within the specified time limit. After training, the learned policy is evaluated by repeatedly calling the `get_action` function.

3.3 Submission Format

This assignment is to be done individually or in pairs. The choice is entirely yours. If the assignment is being done in pairs then the partner is to be selected only within your class.

- Please submit only your best algorithm for each part.
- Please succinctly describe the core ideas of your algorithms in the text file called `report.txt` (upto 2 pages in standard 11 point Arial font). The first line of the report should list the name and entry number of student(s) who submit the assignment.
- The assignment is to be submitted on Gradescope. **Exactly ONE of the team members** needs to make the submission. All registered students have already been added to Gradescope. The details are given below.
 - You need to upload a single ZIP file named `submission.zip`. Upon unzipping, this zip should create a folder containing two folders `part_a`, `part_b`, and two file `report.txt` and `group.txt`. Your agent code for part A should be in file `part_a/agent.py`. Similarly your code for part B agent should be in `part_b/agent.py`. Do not rename the zip file.
 - The `group.txt` should include the entry numbers of all the group members (one per line). If you are working alone, simply include your own entry numbers. A sample ‘`group.txt`’ is given below.

- Gradescope will run a sanity check for your submission. Sanity check will test your code for correct input/output formats and illegal imports and will run your code with dummy inputs. For part A it runs the test case 3 with time budget as 60 seconds and runs part B with time budget 60 seconds. Your submission must pass this sanity test. Submissions that do not clear the sanity check will not be evaluated.
- If you are working in pairs, you **must** select the partner using the **Group Members** option after uploading the submission in Gradescope.

3.4 Evaluation

- Both Part A and Part B will primarily be evaluated based on the *discounted cumulative reward* achieved by the learned policy after being allowed to learn for the specified time. The primary objective is to maximize the quality of the learned policy rather than minimize training time.
- Approximately 80% of the score will be determined by comparing the performance of your algorithm in relation to internal TA baselines. The remaining 20% of the score will be based on how efficiently your algorithm learns when allowed less time budget.
- For Part A, the grid size may be as large as 30×30 . For evaluation with internal baselines in Part A, the maximum training time for an instance with $N = 30$ will be approximately 15 minutes. The TA baseline got a score of ~ 8 with a time budget of 60 seconds on test case 1 and got a score ~ 4.8 with a time budget of 900 seconds on test case 2. For Part B, the maximum training time will also be approximately 4 minutes. The TA baseline got a score of ~ 5.0 in part B.

3.5 Implementation Guidelines and Hints

- The objective of this assignment is to help you gain hands-on experience in implementing and understanding the AI algorithms covered in the course. Our implementations, based on first principles, are able to learn good policies for both domains. These policies allow the agents to exhibit fairly intelligent and intuitive behavior on the given problem instances.
- The following suggestions are intended as starting points and are not exhaustive. For Part A, you can start with Policy Iteration or Value Iteration. You may explore techniques such as Prioritized Sweeping or clustering states into subgroups to further improve the learned policy. For Part B, you can experiment with different exploration–exploitation schedules and learning rates to improve the performance of Q-learning.
- **The suggestions above are not intended to limit your choice of implementation. You are encouraged to explore other algorithms, methods, and design choices that are within the scope of the concepts covered in the course. However, algorithms or methods that are beyond the scope of the class or that have a fundamentally different mathematical underpinning are not intended for this assignment and will not be evaluated.**